

Know your area

Team Vagabonds · Aavaran · problem statement SIH26171

Your part of the project, in a form you can revise in five minutes before a meeting.

Read your own section. Skim the others so you know who to hand a question to.

Everyone should know this

- **The problem statement:** SIH26171, floated by **ISRO**
- **Deadline:** portal closes **30 September 2026**. Finale is **December**
- **What it is:** a **browser extension** for Chrome and Firefox
- **What it does:** lets an AI agent read and act on a web page, while every **PAN, Aadhaar, card number, password and face** is covered up **before anything is sent**
- **The one-line pitch:** the part of our software that talks to the internet has **never held your real data**, so it cannot leak it
- **Headline numbers:** real task done in **about 31 seconds, 23 automated checks** on **3 operating systems**, everything **public and reproducible**

The three questions everyone gets

- **"What is the project?"** An AI agent has to see your screen to help you. Your screen has your PAN and Aadhaar on it. Every existing agent sends a photo of the whole screen to a company server. We cover up the private parts first.
- **"Why not just blur it?"** An agent that cannot read a field cannot act on it either. Blur the PAN and it no longer knows whether the box is filled, which is the fact it needs. We replace each value with a **label** saying what kind it is and that it is valid. The agent gets the fact, not the value.
- **"Tools like this already exist."** They do, and they all run **on the server**, where the data has already arrived. Our contribution is moving detection **into the browser**, the last place where refusing to send is still possible.

Harsh Bajpai

Architecture and the redaction engine

In one line: I worked on where the data is allowed to go, and the part that finds private values and covers them up.

What it is

- A browser extension has **two halves that cannot see into each other**
- The **content script** runs inside the page and holds the real values
- The **background worker** is the only part allowed to touch the network
- Real values stay in the first half. The second only ever gets **labels** like `<PII_PAN_1>` plus a picture with private parts painted over
- The engine works in **three layers**:
 - **Patterns** catch fixed shapes, e.g. a PAN is 5 letters, 4 digits, 1 letter

- **Checksums** catch mathematical structure, telling an Aadhaar from a 12-digit order number
- **Context** reads the field label and surroundings, which is how **names** are caught,

since a name has no shape or checksum

Why it was done this way

- We could have written one careful program. Instead we split it so the part that **could** leak never **has** anything to leak
- That turns a promise about code quality into a **fact about where data sits**
- A bug in the networking half tomorrow still cannot leak a PAN. It has never held one

What went wrong

- **Five separate times**, something was visible on screen but invisible to the redactor, while the screenshot sent it anyway
- Hidden text. **Shadow DOM**, a part of a page ordinary code cannot see into. Text past an internal length limit. A value appearing twice, covered in one place only
- **The worst one:** the masking step had only ever been given **face** positions and had never covered any text at all. A PAN could be perfectly labelled in the data and perfectly readable in the picture sent beside it

Numbers to know cold

- Context layer is worth **8 points of recall: 91.7% without it, 100% with it**
- More usefully: **without it one value leaks, with it none do**

Q and A

- **"How do you know it does not leak?"** Two automatic checks. One searches the entire outgoing message for every value we hid. The second checks the **picture agrees with the data**, which exists because for a while it did not.
- **"What if the AI provider is malicious?"** Does not matter. They receive labels. There is nothing to take.
- **"What are you not defending against?"** Anyone who **already controls the machine**. A compromised browser or a malicious extension update defeats it completely. Section 4.9 says so.
- **"Why not do redaction on the server?"** The data has already crossed the network by then. That concedes the whole point.

Do not claim

- That it is unbreakable, or that it stops malware
- That 100% means it never fails. The report says **twelve values is a small sample**

Go and look at: Figure 1, page 4. The dashed line is the entire architecture.

Vansh Khosla

The repository, the builds, and making it run elsewhere

In one line: I look after the repository and the builds, and the testing that proves it runs on machines that are not ours.

What it is

- Code has to be **packaged** into something each browser accepts, and **Chrome and Firefox want different things**
- **Continuous integration**: every code change is downloaded fresh and the whole suite runs on **Windows, macOS and Linux** automatically
- The **capture path** is also here: screenshot the page, hand it to a hidden part of the extension that can do image work, **paint solid blocks** over private regions, encode it

Why it was done this way

- We test from a **fresh clone**, a clean download, not the copy on the machine where the code was written
- Sounds pedantic. It caught **two bugs** that existed only because a developer's machine had something installed
- Testing on **real Windows** caught **five more**, including a file path hardcoded to one person's Desktop

What went wrong

- A timing test asserted something must finish **under six seconds**
- It failed twice on commits that changed **only documentation**, because the machine was busy and it took **6,021 ms**
- **A fixed time limit is a claim about the computer, not the code**
- It now measures a reference operation on the same machine and compares. We proved the relaxed version still catches a real slowdown by **deliberately introducing one**

Numbers to know cold

- **24 checks** total, **17 need no browser** and run in about a minute
- **3 operating systems**, **2 tagged releases**, setup is **one command**

Q and A

- **"How do you know it works on other machines?"** Built and tested on three operating systems automatically, on every change.
- **"What does someone need to install?"** Node and Python for tests. Running the agent needs a **6 GB** model download; building and testing does not.
- **"Is the code available?"** Yes. Public repository, no account, and **one command reproduces every number** in the report.
- **"Why two browsers?"** The problem statement asks for Chrome and Firefox, and they differ enough that supporting both is real work.

Do not claim

- That it is tested on every browser. It is **Chrome and Firefox**
- That CI catches everything. It catches **what we thought to test**, which is why Section 5 exists

Go and look at: run `./test-all.sh` on your own laptop. If it fails there, that is a real finding worth reporting.

Manas Bharadia

Evaluation, and why any of our numbers mean anything

In one line: I work on testing and measurement, and on making sure our scores are honest rather than flattering.

What it is

- Realistic **test pages**: Indian checkout, bank statement, government form, pages in **Hindi, Tamil and Bengali**, a shadow-DOM page, a page with an embedded frame
- Each has a hand-written **answer key** listing which values are private and which only look private
- A **scoring program** runs the software over every page and marks it against the keys
- **Three sets, and the distinction matters:**
 - **11 tuned pages** we developed against
 - **2 held-out pages**, written before the first scoring run, **never tuned against**
 - **4 real captured websites**, where the page structure is **not ours at all**

Why it was done this way

- The **held-out rule** is the whole point. The finale reveals its test cases **on the day**
- A system tuned to its own tests predicts **nothing** about December
- We treat special-casing a held-out page as **destroying evidence**, not fixing a bug

The thing that will impress her if you get it right

- Our headline scores are **100%**, and the report treats that **with suspicion in its own results**
- On the held-out set the denominator is **twelve values**
- By a standard statistical rule, **no failures in twelve attempts** is still consistent with a true failure rate as high as **25%**
- So the honest reading is "**it did not fail on anything it was shown**", not "it rarely fails"
- We wrote that ourselves, because a professor would otherwise think it for us

What went wrong

- The third corpus, built from **real websites**, found a leak on its **very first run**
- Three earlier test sets had missed it because they were all small enough to fit inside a size limit the real pages exceeded

Numbers to know cold

- **11 tuned / 2 held out / 4 real** pages
- **52 private values** on tuned, **12** on held-out
- Recall **100%** on both, **75%** on the real-page set
- **Zero leaks** everywhere

Q and A

- "**Did you write your own tests?**" Yes, and the report says so plainly. That is exactly why we built the third set from real pages.

- **"Why is one score 75%?"** On the harder set, content below a size limit is never examined, so three values are missed. It is **item 1 in our plan**.
- **"What is a decoy?"** A value that **looks** private but is not. A 12-digit order total that passes the Aadhaar checksum by chance. Covering it up would be an error, and we are scored on that too.
- **"Is the corpus big enough?"** No, and we say so. Growing it is in the plan.

Do not claim

- That the corpus is large. It is not, and the report admits it
- That 100% means it always works

Go and look at: `bench/pages/checkout.truth.json`. That is an answer key, readable without any coding knowledge, and it makes the whole idea concrete in a minute.

Jinshri Jain

The Privacy Ledger, and making the claim checkable

In one line: I work on the screen that shows what was hidden and what was actually sent, so a user can check our privacy claim instead of taking our word for it.

What it is

- A screen with **two columns**: left is what was **on your screen**, right is what **actually left your computer**
- Covered rows show the **label** sent in place of the value
- Harmless rows show the value going out **unchanged**
- **Top**: a count of how many values were withheld and of what kinds
- **Bottom**: a link to the **exact bytes transmitted**, so a sceptic can read the raw message

Why it was done this way, and this is the interesting bit

- The left column **does not store your real values**. It reconstructs a **masked shadow**
- You see enough to recognise which row is which, and **not enough to be worth stealing**
- A screen that kept a permanent record of every secret **in order to prove it had protected them** would be worse than the problem it solves
- Every privacy product makes a claim. Almost none **let you check it**
- It only works if a **non-engineer** can read it in seconds. That is a design problem before it is a coding one

Numbers to know cold

- On the checkout page it withholds **10 values** across **9 kinds**: name, 2 emails, phone, PAN, Aadhaar, GSTIN, card, password, address
- **3 values** are correctly judged **harmless** and sent unchanged: order number, total, tracking number

Q and A

- **"What am I looking at?"** Left is your screen, right is what was sent. **Red** is covered up, **green** is the label that replaced it.

- **"Why show the harmless values too?"** Because a system that hides everything is useless. Showing the order number go out unchanged is **evidence of judgement**.
- **"Could the ledger itself leak?"** No, and that was deliberate. It never holds the real values.
- **"Who is it for?"** A user who wants to verify, and a judge who wants to check the claim in ten seconds rather than read our code.

Do not claim

- That you designed **the redaction**. You designed **how it is shown**. That is Harsh's area and the distinction is easy to hold

Go and look at: Figure 2, page 7. Read it as a stranger and write down the first thing that confuses you. That note is worth more to us than agreement.

Aarna Chauhan

The deck, the diagrams, and how the argument reads

In one line: I do the deck, the diagrams and how the report presents itself.

What it is

- **Three things** carry this project to people who will never read the code: the **six-slide deck**, the **diagrams**, and the **layout of the report**
- **Figure 1** is one dashed line, real data on one side, the internet on the other
- **Figure 2** is the ledger
- Everything else in thirteen pages hangs off those two pictures

Why it was done this way

- Most projects open with an architecture diagram of **twelve boxes that explains nothing**
- Ours has **three boxes and a line**, because **the line is the idea**
- If somebody understands only that dashed line, they understand the project

What is worth understanding about the report

- It is **written to be attacked**
- Reports against **ISRO's published marking scheme**, not measures we chose
- Gives the **denominator** behind every score, because a percentage without one is not a measurement
- Has a section called **What we have not proved**
- States a score of **75% in bold** rather than burying it
- **Why:** a report where everything is perfect reads as a report whose tests were too easy, and a reader who suspects that **stops believing all of it**. One honest weak number makes every other number credible

Numbers to know cold

- ISRO's marking scheme: **25%** visual context, **20%** catching private data, **20%** not over-covering, **20%** light on the machine, **15%** task latency

- The report is **13 pages** with **11 references**

Q and A

- **"Why does the report admit so much?"** Because the alternative is a document a researcher does not believe.
- **"What does Figure 1 show?"** Where data is allowed to go. Left of the line holds real values; right of it has **only ever seen labels**.
- **"Who is the report for?"** Someone deciding whether to spend one of the institute's nomination slots on us, who needs to **check** our claims rather than take them on faith.
- **"Why is the ledger screenshot so dark?"** It is the real product interface, not a mock-up. Recolouring it would make it a marketing render in a document that argues nothing is staged.

Do not claim

- **Authorship of the measurements.** Your area is how the argument is presented, which is a real contribution and does not need inflating

Go and look at: page 5, the results table. The Weight column matches ISRO's marking scheme exactly, and every cell carries a denominator. Both deliberate.

Siddhartha Chaudhary

The demonstration and the Indian-language coverage

In one line: I work on the demo and the runbook, and on the Indian-language coverage, because a word we are missing in Gujarati costs us marks.

What it is

- **The demo:** load a page carrying private data, give the agent an instruction, let it work, then show the ledger
- **The runbook:** the printed sheet saying exactly what to click, in what order, and what to do when something breaks
- **The word lists:** ordinary commercial words in **eleven Indian languages**. Words like total, amount, receipt, bill, price

Why the word lists matter, which is not obvious

- A **12-digit total** on a bill can pass the **Aadhaar checksum purely by chance**
- The only thing stopping the software covering up that total is **recognising the word next to it**
- If the Gujarati word for "total" is missing, a **harmless number gets hidden**
- Hiding harmless things is worth **20% of the marks**, exactly as much as missing a private one
- **Eight of the eleven languages** have still not been checked by someone who reads the script

Why the demo is built this way

- It uses a **multi-step task**, not a single click: choose a category, type a reference, write a description, submit
- The submit button **stays disabled until all three fields are filled**, so it cannot be faked with one lucky click

- A demo that only ever does one thing **proves almost nothing**

What went wrong

- For a long time the automatic check on the demo passed **whenever a result file existed**. It never read what was in it
- So the test reported success on **every run where the agent filled two fields of three and gave up**
- Reading the page afterwards instead exposed **three real defects in a row**

Numbers to know cold

- Task takes a **median 31.5 seconds** over **6 turns**, and completes in **7 runs out of 8**
- **11 languages, 8 unreviewed**
- Model is **Qwen2.5-VL 7B**, about **6 GB**, running **locally with no internet**

Q and A

- **"Can I see it work?"** Yes, and on a laptop with **no internet connection**, which is the quickest way to see the privacy claim is structural rather than a promise.
- **"How long does it take?"** About **half a minute** for a three-field form, slower than doing it by hand. The report says so: today it earns its place **where privacy matters more than speed**.
- **"Why eleven languages?"** It is an India-facing product and **government portals are not in English**.
- **"What happens if the model gets it wrong?"** The extension **validates every action** before running it, and refuses anything that names an element that does not exist.

Do not claim

- That **all eleven languages are verified**. Eight are not, it is in the report, and this is the one claim here somebody could actually check

Go and look at: RUNBOOK.md, then run the demo once. The number of things that go wrong the first time is exactly why the runbook exists.

If you are asked something you do not know

- Say so and **hand it over**. "That is Manas's area, let me get you a proper answer" costs nothing
- Nobody expects six people to each know everything
- **Guessing is what costs you**. A confident wrong answer about your own area is the one thing that makes a reader doubt the whole document
- This report works precisely because it is **careful about what it claims**